

CodeSentinel Scan Report

Summary

MediVault — a healthcare management application (Node.js/Express backend, React/Vite frontend). The codebase is explicitly labeled as a demo for intentional vulnerabilities, but the scan surfaces every real issue with full evidence. The codebase contains **hardcoded production database credentials in both source code and git history**, **plaintext password storage**, **completely unprotected API routes** (no auth middleware), **information-disclosure error handling**, **hardcoded JWT secrets with a signing/verification mismatch**, **Stripe keys leaked in git objects**, and **default weak credentials** for all seed users. This application is critically insecure and must never be deployed as-is.

Repository Facts

Property	Value
Application	MediVault (Healthcare Management)
Backend	Express.js v5 on port 5000
Frontend	React + Vite (SPA)
Database	PostgreSQL (with in-memory fallback)
Auth library	jsonwebtoken (with base64 fallback)
Default seed passwords	password123 for all 31 users
Auth middleware on routes	None — all CRUD routes are fully public
Git author	Neverask1121 <rpsh2823@ramadevipublicschool.com>
Semgrep	0 findings (configs: p/ci, p/security-audit, p/owasp-top-ten)

Secret Scan Result

TruffleHog found **30 findings**. After removing `node_modules` noise (25 false positives from library READMEs/docs in `axios`, `keyv`, `json5`, `zod`), **5 actionable findings** remain across 2 categories:

🔴 CRITICAL — Hardcoded Postgres Credentials in Source Code

Finding	File	Detail
db.js:8 (filesystem)	backend/db/db.js	postgres://medivault_admin:mv_prod_db_sec_7a2b9c@localhost:5432/medivault — username medivault_admin, password mv_prod_db_sec_7a2b9c. Verified . Used as fallback when DATABASE_URL env var is unset.
unsafeModeNotice()	backend/db/db.js:39-46	This credential is exposed via the public /api/health endpoint in its response body.

🔴 CRITICAL — Git History Secret Leak (Persists Forever)

Commit	File	Credential	Host
bbc9e91 (git)	backend/db/db.js	postgres://postgres:wd8aooUc6yfFMiBYsfVVeojZ7FeJcwiM0b9eJZuIaJvYvddYJHNqvMW03wg4LdQo@51.38.51.147:5927/postgres	External IP 51.38.51.147:5927
a897b44	backend/db/db.js	Replaced with the dummy medivault_admin credential — but the real credential above remains in git history permanently	—

The original production credential (password `wd8aooUc6yfFMiBYsfVVeojZ7FeJcwiM0b9eJZuIaJvYvddYJHNqvMW03wg4LdQo`) targets an **external IP address** (`51.38.51.147:5927`), strongly suggesting this is a real production database, not a local test instance. It was committed by `Neverask1121 <rpsh2823@ramadevipublicschool.com>` .

🔴 HIGH — Stripe Keys in Git Objects

Git Object	Detector
49cf60e248e... (BASE64)	Stripe API key
df4ff1789b8... (BASE64)	Stripe API key
fe8670de282... (BASE64)	Stripe API key

These are Base64-encoded Stripe keys stored in git object blobs. Even if no longer referenced in the working tree, they persist in `.git/objects` and are recoverable.

🟢 Excluded — False Positives (node_modules)

25 findings in `node_modules/` (`axios` README URI examples, `keyv` Redis/MongoDB connection examples, `json5` GitHub OAuth examples, `zod` test URIs). All are library documentation/test fixtures, not actual secrets.

Static Analysis Result

Semgrep completed with **0 findings** across rulesets `p/ci`, `p/security-audit`, `p/owasp-top-ten`. This is because the codebase uses safe parameterized queries (`$1`, `$2`, etc.) for all SQL operations, preventing SQL injection. However, Semgrep did not flag the application-level architectural vulnerabilities identified below via manual review.

Findings

Finding 1 – CRITICAL: No Authentication on All API CRUD Routes

Files: `backend/routes/patientRoutes.js`, `backend/routes/recordRoutes.js`, `backend/routes/reportRoutes.js`, `backend/routes/appointmentRoutes.js`

Evidence: None of the route files import or use `authMiddleware`. All routes (`GET /`, `POST /`, `GET /:id`, `PUT /:id`, `DELETE /:id`) are fully public:

```
// patientRoutes.js – no authMiddleware applied
router.get("/", listPatients); // Public
router.post("/", createPatient); // Public
router.get("/:id", getPatientById); // Public
router.put("/:id", updatePatient); // Public
router.delete("/:id", deletePatient); // Public
```

Impact: Any anonymous user can list all patients, create/modify/delete patient records, view medical records, manage appointments, and upload/download reports – **full CRUD access to the entire healthcare database without any authentication**.

Finding 2 – CRITICAL: Plaintext Password Storage and Comparison

Files: `backend/controllers/authController.js:47`, `backend/controllers/seedData.js:7,17,28`

Evidence:

- `bcrypt` is listed in `package.json` but **never imported or used anywhere** in the codebase.
- Login uses direct string comparison: `user.password !== password` (line 47 of `authController.js`).
- All 31 seed users store passwords as plaintext `"password123"`:

```
password: "password123", // seedData.js:7, 17, 28
```

- New user registration stores the raw password: `payload.password` is written directly to the database.

Impact: Any database breach immediately exposes all user passwords in cleartext. Passwords are transmitted and stored without any hashing.

Finding 3 – CRITICAL: Hardcoded JWT Secret with Signing/Verification Mismatch

Files: `backend/controllers/authController.js:16`, `backend/middleware/authMiddleware.js:18`

Evidence:

```
// authController.js (signing)
jwt.sign(..., process.env.JWT_SECRET || "demo-secret");

// authMiddleware.js (verification)
jwt.verify(token, process.env.JWT_SECRET || "mv_prod_jwt_secret_9a8b7c6d5e4f3g2h1i");
```

The fallback secrets **don't match** (`"demo-secret"` vs `"mv_prod_jwt_secret_9a8b7c6d5e4f3g2h1i"`). When `JWT_SECRET` env var is not set, tokens will be signed with one secret and verified with another, causing **all authentication to break**.

Additionally, neither fallback is set in `.env` (which only contains `DATA_MODE` and `PORT`).

Impact: If `JWT_SECRET` is not configured, the auth system is completely non-functional. If it is configured, both hardcoded fallback secrets are exposed in source code.

Finding 4 – CRITICAL: Unsafe Base64 Token Fallback

File: `backend/middleware/authMiddleware.js:20`, `backend/controllers/authController.js:20`

Evidence:

```
// When jsonwebtoken is missing:
// Issue: authController.js:20
return Buffer.from(JSON.stringify({ id, email, role })).toString("base64");

// Verify: authMiddleware.js:20
req.user = JSON.parse(Buffer.from(token, "base64").toString("utf8"));
```

Impact: Without `jsonwebtoken`, authentication tokens are simply base64-encoded JSON – **zero cryptographic protection**. Any user can forge a token with any role (including `admin`) by base64-encoding `{ "id": 999, "email": "admin@medivault.com", "role": "admin" }`.

Finding 5 – CRITICAL: Health Endpoint Leaks DB Credentials

Files: `backend/server.js:24-27`, `backend/db/db.js:39-46`

Evidence:

```
// server.js
app.get("/api/health", async (req, res) => {
  await ensureSeed();
  res.json({ ok: true, ...unsafeModeNotice() });
});

// db.js - unsafeModeNotice()
function unsafeModeNotice() {
  return {
    dataMode,
    database: isPostgresMode() ? "postgres" : "in-memory-fallback",
    note: "INTENTIONAL VULNERABILITY FOR DEMO",
    hardcodedDbCredentialExample, // ← Full DB connection string with credentials
  };
}
```

The public `/api/health` endpoint returns the **full database connection string including username and password** in its JSON response. This endpoint requires no authentication.

Impact: Complete database credential disclosure to any anonymous visitor.

Finding 6 — ❗ HIGH: Information Disclosure via Error Handler

File: `backend/server.js:35-41`

Evidence:

```
app.use((err, req, res, next) => {
  res.status(500).json({
    message: err.message,
    stack: err.stack, // ← Full stack trace exposed to client
  });
});
```

Impact: Full server-side stack traces, file paths, library versions, and error details are leaked to clients, aiding attacker reconnaissance.

Finding 7 — ❗ HIGH: Unrestricted CORS

File: `backend/server.js:17`

Evidence: `app.use(cors())` — no origin restriction, no credentials configuration.

Impact: Any origin can make authenticated cross-origin requests to the API, enabling CSRF-style attacks from malicious websites.

Finding 8 — ❗ MEDIUM: Unrestricted File Upload

File: `backend/routes/reportRoutes.js:11`, `backend/controllers/reportController.js:13-18`

Evidence:

```
const upload = multer ? multer({ dest: "uploads/" }) : { single: () => (req, res, next) => next() };
router.post("/", upload.single("file"), createReport);
```

Multer is configured with only a destination — no file type filter, no size limit. The fallback when `multer` is missing bypasses upload entirely.

Impact: Users can upload arbitrary files (including executables, scripts) to the server.

Finding 9 — ❗ MEDIUM: Weak Default Credentials

File: `backend/controllers/seedData.js`

Evidence: All 31 seed users (20 patients, 10 doctors, 1 admin) use the same password: `password123`. The admin account is `admin@medivault.com` / `password123`.

Impact: Trivial credential guessing for all accounts, including the admin.

Finding 10 — ❗ MEDIUM: Login/Register Responses Leak Full User Object

File: `backend/controllers/authController.js:36-40,51-55`

Evidence:

```
return res.json({
  message: "Logged in",
  token: issueToken(user),
  user, // ← Full user object including plaintext password
});
```

Impact: The login and registration endpoints return the complete user record, which includes the plaintext `password` field in the JSON response.

Dynamic Analysis

No dynamic analysis (browser, JS analyzer, HTTP probing) was performed. This is a **local-only codebase scan** with no deployed URL. The application is not running on a publicly accessible server, so runtime verification is not applicable.

Recommendations

Immediate (P0) — Must Fix Before Any Deployment

1. **Remove hardcoded credentials from source code.** Replace `hardcodedDbCredentialExample` in `db.js` with `process.env.DATABASE_URL` only, with no fallback to a credential string. Throw an error if the env var is missing.
2. **Purge git history.** Use `git-filter-repo` or BFG Repo-Cleaner to remove the real Postgres credential (`wd8aooUc6yffFMiBYsfVVeoJZ7FeJcwiM0b9eJJZuIaJvYVddYJHNqvMW03wg4LdQo`) from all historical commits. **Rotate the database password** immediately — it must be considered compromised.
3. **Rotate all leaked Stripe keys** found in git objects. Regenerate them in the Stripe dashboard.
4. **Add authentication middleware to all routes.** Apply `authMiddleware` to every route except `/api/auth/login` and `/api/auth/register` .
5. **Hash passwords with bcrypt.** Import and use `bcrypt` for both registration (hash) and login (compare). Migrate existing plaintext passwords.
6. **Fix the JWT secret mismatch.** Use a single strong secret (or asymmetric keys), configured exclusively via environment variables, with no hardcoded fallbacks. Throw on startup if `JWT_SECRET` is not set.
7. **Remove the base64 token fallback.** Require `jsonwebtoken` as a hard dependency. Remove the `try/catch` import pattern that silently falls back to base64.

High Priority (P1)

8. **Remove credential disclosure from `/api/health`.** Remove `unsafeModeNotice()` or strip `hardcodedDbCredentialExample` from the response.
9. **Fix the error handler.** Never expose `err.stack` to clients in production. Log server-side only.
10. **Restrict CORS** to the specific frontend origin (e.g., `cors({ origin: 'http://localhost:5173' })`).
11. **Strip the `password` field** from all API responses before sending to clients.

Medium Priority (P2)

12. **Add file upload validation** — restrict allowed MIME types and file size limits in multer configuration.
13. **Replace weak seed passwords** with randomly generated values, and provide a proper setup/seeding script.
14. **Add rate limiting** to auth endpoints to prevent brute-force attacks.

Limitations & Next Steps

- **No dynamic analysis** was performed because the application is not deployed to a public URL. Runtime verification (cookie inspection, WebSocket analysis, XSS testing) was not applicable.
- **Semgrep produced 0 findings** across standard rulesets. This is expected because the codebase uses parameterized queries (no SQL injection) and the architectural vulnerabilities (missing auth, plaintext passwords, credential disclosure) fall outside typical SAST rule patterns. A custom ruleset targeting auth bypass and credential exposure patterns would yield results.
- **Git object Stripe keys** could not be decoded to determine their validity or type (test vs. live). They should be assumed compromised and rotated.
- **Authenticated Semgrep AppSec Platform scans, deployed URL scans, and PR creation** are intentionally out of scope for local scans in the current phase.
- The database at `51.38.51.147:5927` may still be live. If this is a production system, the credential leak in git history represents an **active security incident** requiring immediate credential rotation and access audit.